Key Matrix Application Note

Revision History

Date	Version	Description
September 29, 2021	0.50	Initial version created.

CONFIDENTIAL

©2021, Atmosic Technologies Inc. All rights reserved. Atmosic logo is a registered trademark of Atmosic Technologies Inc. All other trademarks are the properties of their respective holders. This preliminary document is subject to change without notice.

Table of Contents

1 Overview 4

2 Process Flow 4

2.1 Enable Keyboard and atm_vkey 5

2.2 Key Matrix Configuration 5

2.2.1 Key scan matrix 5

2.2.2 Pin MUX options for KSI/KSO 6

2.2.3 Key matrix setting 7

2.2.3.1 Default setting 7

2.2.3.2 Customized setting 9

2.2.3.3 Customized setting example 10

3 Virtual Key Mapping 11

3.1 Virtual Key Index 11

3.1.1 Conversion example 11

3.2 Key Code Table 11

3.2.1 HID_remote Virtual Key Tables 11

3.2.2 HID_keyboard virtual key tables 12

4 Virtual Key Handling 12

4.1 Virtual Key Table 13

4.2 Setup Flow 14

CONFIDENTIAL

List of Figures

Figure 1 - Process Flow
Figure 2 - Key Scan Matrix
Figure 3 - Setup Flow

List of Tables

Table 1 - All possible KSI and KSO MUX Settings
Table 2 - Default Setting
Table 3 - Row Configuration
Table 4 - Column Configuration

CONFIDENTIAL

1 Overview

This application note shows how to configure and match a key matrix with keyboard driver and virtual key settings in HID_remote and HID_keyboard examples according to the exact hardware design.

2 Process Flow

Key matrix and virtual key setup steps:

1. Enable keyboard and atm_vkey
2. Key matrix configuration
3. Virtual key mapping
4. Virtual key handling

Figure 1 depicts the key event handling flow with keyboard driver and atm_vkey module.

Keyboard: key scan matrix hardware driver.
Atm_vkey: framework module which handles key behavior such as click,hold..etc.
rc_mmi_vkey: application module for implement related handler and link keyboard driver and atm_vkey

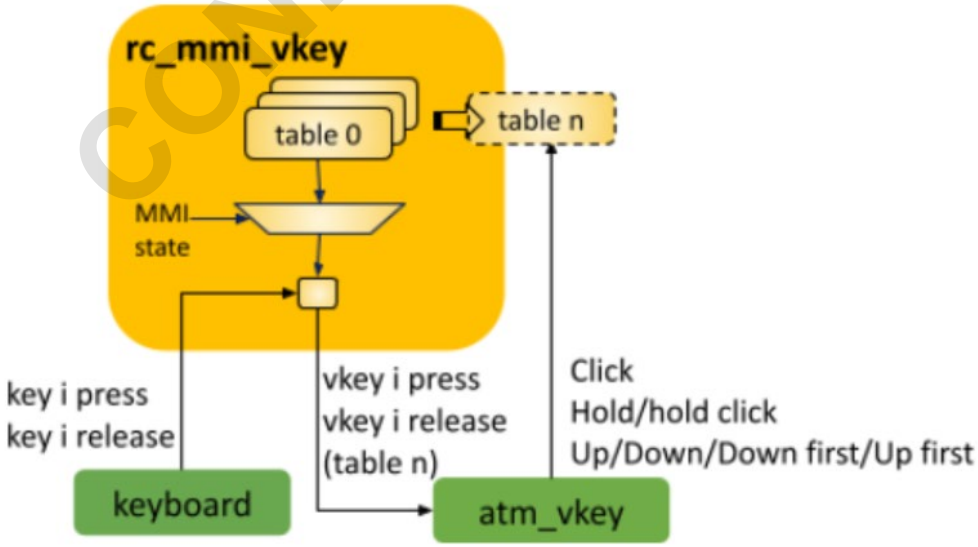


Figure 1 - Process Flow

CONFIDENTIAL

2.1 Enable Keyboard and atm_vkey

Add **keyboard, atm_vkey, sw_event** for DRIVERS and **CFLAGS += -DCFG_KBD -DKSI_PULLUPS** in makefile

HID_remote makefile

```
DRIVERS := \
    atm_ble \
    atm_gpio \
    atm_pm \
    atm_vkey \
    gadc \
    interrupt \
    keyboard \
    led_blink \
    pdm \
    sw_event \
    sw_timer \
    trng \

# Keyboard
CFLAGS += -DCFG_KBD -DKSI_PULLUPS
```

HID_keyboard makefile

```
DRIVERS := atm_ble atm_pm gadc interrupt keyboard trng sw_timer timer sw_event
atm_gpio atm_vkey
# Drivers: KSM
CFLAGS += -DCFG_KBD -DIGNORE_P18P19 -DKSI_PULLUPS
```

2.2 Key Matrix Configuration

2.2.1 Key scan matrix

Figure 2 depicts a 4 rows by 4 column **KSM** (key scan matrix). It can support up to 16 keys detection on each cross with the key scan mechanism. The key scan signal would be generated

CONFIDENTIAL

from the column pins which are set MUX for **KSO** (key scan output) and the key scan result would be collected at the row pins which are set MUX for **KSI** (key scan input).

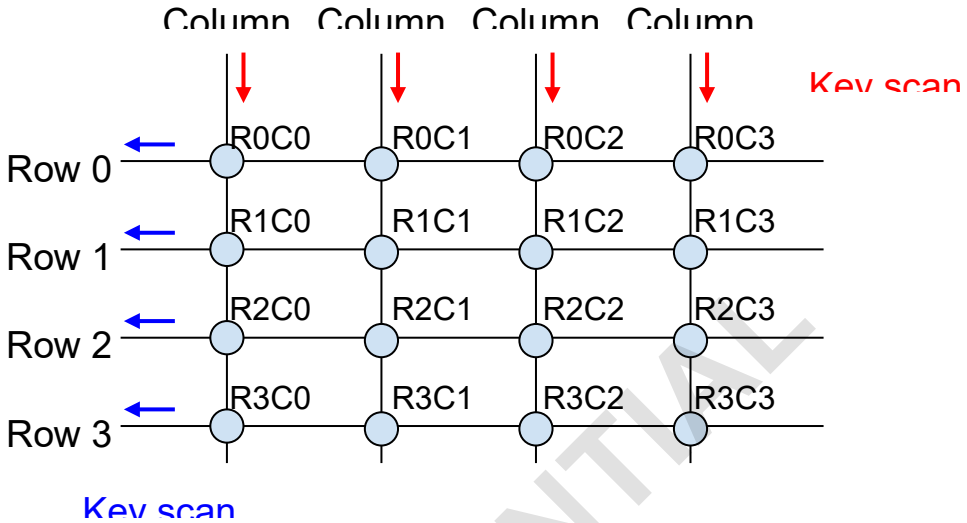


Figure 2 - Key Scan Matrix

2.2.2 Pin MUX options for KSI/KSO

Table 1 shows all possible KSI and KSO MUX settings for ATM2/3 chips.

Pin	Key Scan Input	Key Scan Output Alternative 1	Key Scan Output Alternative 2	Key Scan Output Alternative 3
P0	KSI_19	KSO_19	KSO_8	-
P1	KSI_18	KSO_18	KSO_9	-
P2	KSI_17	KSO_17	KSO_10	KSO_8
P3	KSI_16	KSO_16	KSO_11	KSO_9
P4	KSI_15	KSO_15	KSO_12	KSO_10
P5	KSI_14	KSO_14	KSO_13	KSO_11
P6	KSI_13	KSO_13	KSO_14	KSO_12
P7	KSI_12	KSO_12	KSO_15	KSO_13
P8	KSI_11	KSO_11	KSO_16	KSO_14
P9	KSI_10	KSO_10	KSO_17	KSO_15

CONFIDENTIAL

P10	KSI_9	KSO_9	KSO_18	KSO_16
P11	KSI_8	KSO_8	KSO_19	KSO_17
P12	KSI_7	-	-	KSO_18
P13	KSI_6	-	-	KSO_19
P18	KSI_5	-	-	-
P19	KSI_4	-	-	-
P20	KSI_3	-	-	-
P21	KSI_2	-	-	-
P22	KSI_1	-	-	-
P23	KSI_0	-	-	-
P24	-	-	KSO_7	-
P25	-	-	KSO_6	-
P26	-	-	KSO_5	-
P27	-	-	KSO_4	-
P28	-	-	KSO_3	-
P29	-	-	KSO_2	-
P30	-	-	KSO_1	-
P31	-	-	KSO_0	-

Table 1 - All possible KSI and KSO MUX Settings

CONFIDENTIAL

2.2.3 Key matrix setting

There are two setting types below to configure the key matrix. Users can choose either one to complete their key matrix configuration.

- Default setting
- Customized setting

2.2.3.1 Default setting

The default setting is designed for only supplying NUM_ROWS and NUM_COLS macros to achieve wide range support up to 20 columns (8 by 20) or 20 rows numbers (20 by 8). The default

setting macros are defined through ***CFLAGS += -DNUM_ROWS=n -DNUM_COLS=m*** in makefile. Please refer to driver/keyboard/keyboard_param.h for more details.

Table 2 shows the default settings.

In the default setting, the first 8 row and column pins (P23~P12, P31~P24) are dedicated and the other pins would be configured as either KSI or KSO according to NUM_ROWS and NUM_COLS. To use the default setting, the hardware design must follow the corresponding pin assignment. The pin assignment cannot be changed in the default setting. Default setting is recommended when the hardware design is flexible to reserve required KSI/KSO pins and layout them in order. If the hardware design is already fixed and some required pins are used for other purposes, the user has to use the customized setting to configure the key matrix.

CONFIDENTIAL

Row #	Pin	KSI	Column #	Pin	KSO
0	P23	KSI_0	0	P31	KSO_0
1	P22	KSI_1	1	P30	KSO_1
2	P21	KSI_2	2	P29	KSO_2
3	P20	KSI_3	3	P28	KSO_3
4	P19	KSI_4	4	P27	KSO_4
5	P18	KSI_5	5	P26	KSO_5
6	P13	KSI_6	6	P25	KSO_6
7	P12	KSI_7	7	P24	KSO_7
8	P11	KSI_8	8	P0	KSO_8
9	P10	KSI_9	9	P1	KSO_9
10	P9	KSI_10	10	P2	KSO_10
11	P8	KSI_11	11	P3	KSO_11
12	P7	KSI_12	12	P4	KSO_12
13	P6	KSI_13	13	P5	KSO_13
14	P5	KSI_14	14	P6	KSO_14
15	P4	KSI_15	15	P7	KSO_15
16	P3	KSI_16	16	P8	KSO_16

17	P2	KSI_17	17	P9	KSO_17
18	P1	KSI_18	18	P10	KSO_18
19	P0	KSI_19	19	P11	KSO_19

Table 2 - Default Setting

4 x 4 key matrix configuration in default setting:

```
CFLAGS += \
    -DNUM_ROWS=4 \
    -DNUM_COLS=4 \
```

2.2.3.2 Customized setting

The customized setting is used when the default setting can't meet the user's requirement. Users can define all the *ROWi/ROWi_KSI* and *COLj/COLj_KSO* in the makefile to configure the desired key matrix they need. DO NOT define *NUM_ROWS* or *NUM_COLS* in customized settings.

4x4 key matrix configuration in customized settings is shown in Table 3 and Table 4.

Row	0	1	2	3
Pin	23	22	20	13
KSI	0	1	3	6

Table 3 - Row Configuration

Column	0	1	2	3
Pin	30	11	10	9
KSO	1	8	9	10

Table 4 - Column Configuration

```
CFLAGS += \
    -DROW0=23 -DROW0_KSI=0 \
    -DROW1=22 -DROW1_KSI=1 \
    -DROW2=20 -DROW2_KSI=3 \
    -DROW3=13 -DROW3_KSI=6 \
    -DCOL0=30 -DCOL0_KSO=1 \
    -DCOL1=11 -DCOL1_KSO=8 \
```

CONFIDENTIAL

```
    -DCOL2=10 -DCOL2_KSO=9 \
    -DCOL3=9 -DCOL3_KSO=10 \
```

2.2.3.3 Customized setting example

User's requirement:

Row	0	1	2	3	4	5	6	7
Pin	23	22	21	20	10	11	12	9
KSI	0	1	2	3	9	8	7	10

Column	0	1	2	3	4	5	6	7
Pin	31	30	29	28	3	4	5	6
KSO	0	1	2	3	16	15	14	13

8 x 8 key matrix configuration:

```
CFLAGS += \
    -DROW0=23 -DROW0_KSI=0 \
    -DROW1=22 -DROW1_KSI=1 \
    -DROW2=21 -DROW2_KSI=2 \
    -DROW3=20 -DROW3_KSI=3 \
    -DROW4=10 -DROW4_KSI=9 \
    -DROW5=11 -DROW5_KSI=8 \
    -DROW6=12 -DROW6_KSI=7 \
    -DROW7=9 -DROW7_KSI=10 \
    -DCOL0=31 -DCOL0_KSO=0 \
    -DCOL1=30 -DCOL1_KSO=1 \
    -DCOL2=29 -DCOL2_KSO=2 \
    -DCOL3=28 -DCOL3_KSO=3 \
    -DCOL4=3 -DCOL4_KSO=16 \
    -DCOL5=4 -DCOL5_KSO=15 \
    -DCOL6=5 -DCOL6_KSO=14 \
    -DCOL7=6 -DCOL7_KSO=13 \
```

CONFIDENTIAL

3 Virtual Key Mapping

3.1 Virtual Key Index

The key matrix scan result would be converted into a virtual key index with the row and column indexes before the key event is sent to the application from the keyboard driver.

Virtual Key Index (Row i, Column j) = *i * MaxColumn + j*

3.1.1 Conversion example

The key at row 2 and column 3 is detected in a 4 x 4 key matrix. Its virtual key index is:

2 * 4 + 3 = 11

The max vkey amount is defined with **ATM_VKEY_MAX**, and it would be 32 if it is not defined. Need to check the value would be equal or larger than the key matrix.

3.2 Key Code Table

The virtual key index is used to find the corresponding keycode in a key code table. Users can assign the key code value for each key in the key code table. In HID_remote/HID_keyboard example, bt_keycode or scan_codes is used for HID.

The size of the key table must be the same as the key matrix. For the unused keys in the key matrix, users could put zero at the corresponding positions in the table to bypass the handling in mmi vkey implementation.

3.2.1 HID_remote Virtual Key Tables

- src/rc_mmi_vkey_default_x202.h (4 x 4 key matrix)
- src/rc_mmi_vkey_default_x221.h (4 x 4 key matrix)

rc_mmi_vkey_default_x221.h

```
// Virtual key definitions
enum {
    VK_POWER, VK_MIC, VK_UP, VK_RIGHT,
    VK_LEFT, VK_BACK, VK_HOME, VK_MENU,
    VK_OK, VK_RICE, VK_PLAY, VK_VOLUP,
    VK_DOWN, VK_BW, VK_FW, VK_VOLDN,
};
```

CONFIDENTIAL

```
static uint32_t const bt_keycode[] = {
    BT_POWER, BT_MIC, BT_UP, BT_RIGHT,
    BT_LEFT, BT_BACK, BT_HOME, BT_MENU,
    BT_OK, 0, BT_PLAY, BT_VOLU,
    BT_DOWN, BT_BW, BT_FW, BT_VOLD
};
```

3.2.2 HID_keyboard virtual key tables

- src/kbd_mmi_vkey_4_4.h
- src/kbd_mmi_vkey_8_18.h
- src/kbd_mmi_vkey_13_10.h

kbd_mmi_vkey_4_4.h

```
static uint16_t const scan_codes[MAX_ROW * MAX_COL] = {
    KEY_1, KEY_2, KEY_3, KEY_A,
    KEY_4, KEY_5, KEY_6, KEY_B,
    KEY_7, KEY_8, KEY_9, KEY_C,
    KEY_0, KEY_F, KEY_E, KEY_D
};

static char const * const keyarr[MAX_ROW * MAX_COL] = {
    "1", "2", "3", "A",
    "4", "5", "6", "B",
    "7", "8", "9", "C",
    "0", "F", "E", "D"
};
```

CONFIDENTIAL

4 Virtual Key Handling

Please refer to src/rc_mmi_vkey.c for implementation of HID_remote virtual key handling, and src/kbd_mmi_vkey.c for Implementation of HID_keyboard virtual key handling.

4.1 Virtual Key Table

The virtual key table is composed of multiple virtual key events (atm_vk_reg_t). The elements of a virtual key event would be different according to related key actions. The necessary elements are vkey event id, vkey event handling . Other elements would be vkey index, mask, or press time and it would be decided in atm_vk_xxx_reg_t struct .

```
/// Event union for table
typedef struct {
    /// Event id
    atm_vk_eid_t eid;
    union {
      /// Non-first down event
      atm_vk_dnup_inter_reg_t dn;
      /// Non-last up event
      atm_vk_dnup_inter_reg_t up;
      /// First down event
      atm_vk_dnup_reg_t dn_fst;
      /// Last up event
      atm_vk_dnup_reg_t up_lst;
      /// Hold event
      atm_vk_hd_reg_t hold;
      /// Hold and click event
      atm_vk_hc_reg_t hold_click;
      /// click event
      atm_vk_ck_reg_t click;
      /// double click event
      atm_vk_dck_reg_t db_click;
    };
} atm_vk_reg_t;
```

The vkey event id (atm_vk_eid_t) is the type of key action as below.

```
/// Key event ID
typedef enum {
    /// First key down event.
    atm_vk_down_1st,
    /// Non-first key down event. Other pressed key(s) exist.
    atm_vk_down_more,
```

CONFIDENTIAL

```
    /// Key held event.
    atm_vk_hold,
    /// Non-last Key up event. Other pressed key(s) exist.
    atm_vk_up_inter,
    /// Last key up event.
    atm_vk_up_last,
    /// Event for key clicking.
    atm_vk_click,
    /// Event for key clicking with some key(s) held.
    atm_vk_hold_click,
    /// Event for key double clicking.
    atm_vk_db_click,
} atm_vk_eid_t;
```

Example:
Below is the virtual key table for HID_remote example in HID not ready state.

```
// Key events in HID non-ready - VKEY_TABLE_HID_NOT_READY
static atm_vk_reg_t const rc_key_event_hid_not_ready[] = {
    VKEY_DOWN_FIRST(rc_save_key, atm_vk_any),
#ifdef CFG_RC_IR
    VKEY_UP_LAST(rc_save_key_up, atm_vk_any),
#endif // CFG_RC_IR
    VKEY_HOLD(rc_del_bond, 2000, VK_OK_MASK | VK_VOLDN_MASK),
    VKEY_HOLD_CLICK(rc_reset, VK_MENU, VK_FW_MASK | VK_BW_MASK),
    VKEY_HOLD_CLICK(rc_enter_rftest, VK_VOLDN, VK_FW_MASK | VK_BW_MASK),
};
```

There are four vkey events in the table, atm_vk_any means it would be triggered for any virtual key. If any single key was pressed, it would trigger the event and execute the handle - rc_save_key.

4.2 Setup Flow

After preparing the above configuration setting and vkey tables, the application layer needs to register the KSM event callback, initialize the keyboard driver, and add vkey table to atm vkey module.

The KSM event callback function would be executed if there is any state change for the key matrix. In the callback function, it is needed to feed the ksm_event and vkey table to the vkey module, and then the vkey module executes related key handling.

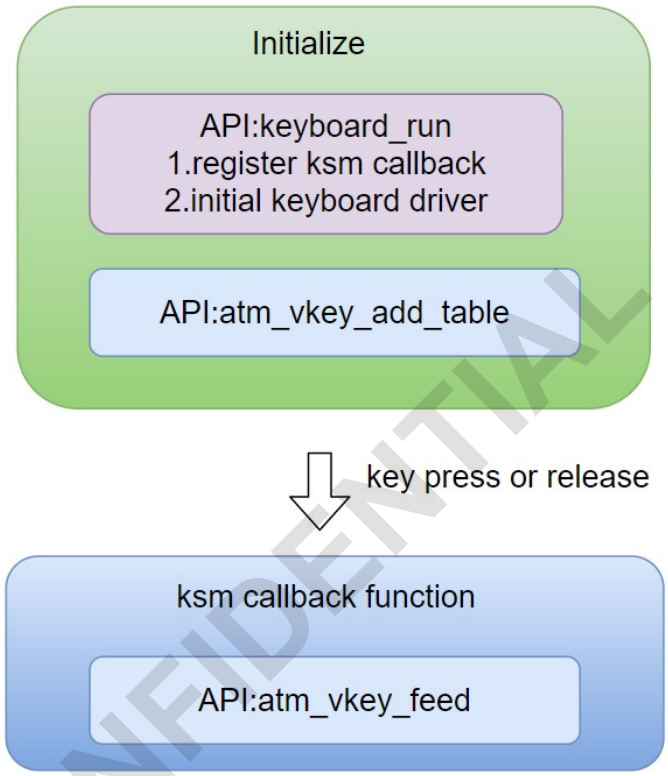


Figure 3 - Setup Flow

CONFIDENTIAL

Example:

In the keyboard example, kbd_mmi_vkeyis the main file to implement key handling. kbd_mmi_vkey_init: Activate the key scan function - keyboard_run and register the callback function - keyboard_callback for KSM event.

```
void kbd_mmi_vkey_init(void)
{
    keyboard_run(keyboard_callback);

    //vkey modeling
    mmi_vkey_ctx.vkey_handles[VKEY_GROUP_DEFAULT] =
atm_vkey_add_table(kbd_unconn_vkeys,
        ARRAY_LEN(kbd_unconn_vkeys), &mmi_vkey_ctx);
    mmi_vkey_ctx.vkey_handles[VKEY_GROUP_HID] =
atm_vkey_add_table(kbd_hid_ready_vkeys,
```

```
        ARRAY_LEN(kbd_hid_ready_vkeys), &mmi_vkey_ctx);
#ifndef CFG_KBD_4_4
    mmi_vkey_ctx.vkey_handles[VKEY_GROUP_PAIRING] =
atm_vkey_add_table(kbd_pairing_vkeys,
        ARRAY_LEN(kbd_pairing_vkeys), &mmi_vkey_ctx);
    mmi_vkey_ctx.vkey_handles[VKEY_GROUP_RFTEST] =
atm_vkey_add_table(kbd_rftest_vkeys,
        ARRAY_LEN(kbd_rftest_vkeys), &mmi_vkey_ctx);
#endif // CFG_KBD_4_4
#ifdef CFG_HID_PASSKEY
    mmi_vkey_ctx.vkey_handles[VKEY_GROUP_PASSKEY] =
atm_vkey_add_table(kbd_passkey_vkeys,
        ARRAY_LEN(kbd_passkey_vkeys), &mmi_vkey_ctx);
#endif
}
```

In keyboard_callback , feed ksm_event and vkey table to the VKEY module.

```
static void keyboard_callback(ksm_event_t evt, uint32_t idx)
{
    if (evt == KSM_ERR_HW_OVF) {
      kbd_mmi_vkey_free_buffer(); // free all saved key
      kbd_hogp_clear_report(); // send all up
      return;
    }

    bool pressed = (evt == KSM_PRESS);
    ATM_LOG(N, "%lu %c S%02x %s", idx, pressed ? 'D' : 'U', scan_codes[idx],
      keyarr[idx]);
    atm_vkey_feed(mmi_vkey_ctx.vkey_handles[mmi_vkey_ctx.cur_vkey_grp], idx,
      pressed);
}
```

CONFIDENTIAL



CONFIDENTIAL

ATMOSIC TECHNOLOGIES - DISCLAIMER

This product document is intended to be a general informational aid and not a substitute for any literature or labeling accompanying your purchase of the Atmosic product. Atmosic reserves the right to amend its product literature at any time without notice and for any reason, including to improve product design or function. While Atmosic strives to make its documents accurate and current, Atmosic makes no warranty or representation that the information contained in this document is completely accurate, and Atmosic hereby disclaims (i) any and all liability for any errors or inaccuracies contained in any document or in any other product literature and any damages or lost profits resulting therefrom; (ii) any and all liability and responsibility for any action you take or fail to take based on the information contained in this document; and (iii) any and all implied warranties which may attach to this document, including warranties of fitness for particular purpose, non-infringement and merchantability. Consequently, you assume all risk in your use of this document, the Atmosic product, and in any action you take or fail to take based upon the information in this document. Any statements in this document in regard to the suitability of an Atmosic product for certain types of applications are based on Atmosic's general knowledge of typical requirements in generic applications and are not binding statements about the suitability of Atmosic products for any particular application. It is your responsibility as the customer to validate that a particular Atmosic product is suitable for use in a particular application. All content in this document is proprietary, copyrighted, and owned or licensed by Atmosic, and any unauthorized use of content or trademarks contained herein is strictly prohibited.

Copyright ©2021 by Atmosic Technologies. All rights reserved.

www.atmosic.com